Onyx Engine

One model.
One process.
OpenAI-ready.



One model per process. Nothing fetched on a request's behalf.

THE PROBLEM

Serving one model should be one command.

01

Glue code

Wrapping an engine in an OpenAI-shaped API is work every app repeats.

02

Queued requests

One request at a time wastes the hardware you already have.

03

Format friction

A Hugging Face model directory isn't a GGUF file, and conversion drags in torch.

WHAT SHIPS

Six things, built in.

Continuous batching

--parallel N slots share one batch per tick, so requests generate together.

Structured output

GBNF grammars, json_schema and response_format.

Tool calls

Jinja chat templates, with parsed and streamed tool-call deltas.

Context shift

A slot that fills its context drops the oldest tokens and keeps going.

Multimodal, no fetching

Image and audio as data: URIs only. The engine performs no I/O for a request.

Safetensors to GGUF

A numpy-only converter turns a Hugging Face Llama directory into GGUF on first load.

ENDPOINTS

Nine routes. That’s the API.

GET /health	GET /props	GET /v1/models
POST /tokenize	POST /detokenize	POST /apply-template
POST /v1/chat/completions	POST /v1/completions	POST /v1/embeddings

SSE streaming ends in a frame carrying usage and timings. /health is exempt from --api-key, so a supervisor can poll it.

BY THE NUMBERS

Small on purpose.

1
model per process

9
HTTP endpoints

4
prebuilt platforms

6
build targets: CPU to ROCm

Remote media URLs are refused with 400, by design.

USE IT

Point any OpenAI client at it.

request

```
$ curl localhost:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"messages": [{"role": "user",
     "content": "Hello!"}]}'
```

Speaks the routes your client already calls:

`/v1/models` `/v1/chat/completions` `/v1/completions` `/v1/embeddings` `SSE streaming`

BUILD TARGETS

CPU by default. GPU when built for it.

CPU	CUDA	Vulkan	Metal	ROCm / HIP	BLAS

Prebuilt release archives, with checksums.txt:

`linux-x64` `darwin-arm64` `darwin-x64` `win32-x64`

Two commands to a served model.

terminal

```
$ npm ci && npm run build
$ ./build/onyx-engine -m model.gguf --port 8080
# add -np 4 for four concurrent slots
```

v0.4.0. Prebuilt for linux-x64, darwin-arm64, darwin-x64 and win32-x64.



Onyx Engine

github.com/JX-Holdings-LLC/Onyx-Engine